

If no collisions occur, `spritecollideany()` returns `None` and the `if` block won't execute ❶. If it finds an alien that has collided with the ship, it returns that alien and the `if` block executes: it prints `Ship hit!!!` ❷. When an alien hits the ship, we'll need to do a number of tasks: delete all remaining aliens and bullets, recenter the ship, and create a new fleet. Before we write code to do all this, we want to know that our approach to detecting alien-ship collisions works correctly. Writing a `print()` call is a simple way to ensure we're detecting these collisions properly.

Now when you run *Alien Invasion*, the message *Ship hit!!!* should appear in the terminal whenever an alien runs into the ship. When you're testing this feature, set `fleet_drop_speed` to a higher value, such as 50 or 100, so the aliens reach your ship faster.

Responding to Alien-Ship Collisions

Now we need to figure out exactly what will happen when an alien collides with the ship. Instead of destroying the ship instance and creating a new one, we'll count how many times the ship has been hit by tracking statistics for the game. Tracking statistics will also be useful for scoring.

Let's write a new class, `GameStats`, to track game statistics, and let's save it as `game_stats.py`:

```
game_stats.py class GameStats:
    """Track statistics for Alien Invasion."""

    def __init__(self, ai_game):
        """Initialize statistics."""
        self.settings = ai_game.settings
        ❶ self.reset_stats()

    def reset_stats(self):
        """Initialize statistics that can change during the game."""
        self.ships_left = self.settings.ship_limit
```

We'll make one `GameStats` instance for the entire time *Alien Invasion* is running, but we'll need to reset some statistics each time the player starts a new game. To do this, we'll initialize most of the statistics in the `reset_stats()` method, instead of directly in `__init__()`. We'll call this method from `__init__()` so the statistics are set properly when the `GameStats` instance is first created ❶. But we'll also be able to call `reset_stats()` anytime the player starts a new game. Right now we have only one statistic, `ships_left`, the value of which will change throughout the game.

The number of ships the player starts with should be stored in `settings.py` as `ship_limit`:

```
settings.py # Ship settings
            self.ship_speed = 1.5
            self.ship_limit = 3
```
